

ADAPTIVE AI CODING MENTOR

Project Documentation

An Agentic Framework for Graduated Programming Assistance and AI-Dependency Assessment

Final Year Project – B.Tech Computer Science and Engineering

Galgotias University

Authors

Agastya Maurya

Nitin Kumar

Under the Guidance of

Professor Vinod Kumar

Document Overview

Adaptive AI Coding Mentor (AACM) is an AI-assisted programming environment designed to help learners solve coding problems without making complete AI-generated solutions the default form of assistance. The system analyzes learner attempts, selects an appropriate level of guidance, generates graduated hints, and records interaction telemetry. It combines agentic components with multiple Large Language Models (LLMs) and provides analytics for programming progress and AI reliance.

1. Project Overview

Project Title: Adaptive AI Coding Mentor: An Agentic Framework for Graduated Programming Assistance and AI-Dependency Assessment. **Domain:** Generative AI, Agentic AI, Programming Education, Intelligent Tutoring Systems, Learning Analytics, and Human–AI Interaction. **Purpose:** The purpose of AACM is to provide the minimum useful AI assistance first and increase guidance when evidence from the learner's attempts indicates that additional support is required.

2. Problem Statement

Conventional AI coding assistants can provide complete solutions immediately. Although this is useful for software development, unrestricted solution generation can reduce the amount of reasoning performed by a student. Completely restricting AI is also undesirable because learners may genuinely need assistance. AACM addresses this balance by treating AI help as a graduated intervention and by recording interaction behavior that can be analyzed over multiple programming sessions.

3. Objectives

1. Develop an AI programming workspace for contextual code assistance.
2. Provide graduated hints from directional guidance to detailed explanations.
3. Adapt assistance using attempts and interaction history.
4. Integrate Qwen, Gemini, and Grok through an LLM routing layer with fallback support.
5. Record attempts, hints, solution requests, response time, model use, and assistance levels.
6. Calculate an AI Dependency Score (ADS) as a project-specific behavioral indicator.
7. Provide dashboards for skill, assistance, and dependency trends.
8. Support controlled evaluation of adaptive versus unrestricted AI assistance.

4. Scope

Included: Programming workspace and editor; AI code analysis; graduated hints; adaptive assistance; Qwen as default model with Gemini and Grok alternatives; model fallback; telemetry; AI Dependency Score; learner analytics and research metrics.

Not included in the initial version: a complete learning-management system, universal automatic grading, treating ADS as a psychological diagnosis, or guaranteeing correctness of unverified LLM output.

5. System Architecture

AACM follows an agentic pipeline: Student → Session Manager → Analyzer Agent → Level Controller → Hint Agent → LLM Router → Response. Telemetry is collected throughout the workflow and is used for learner analytics and dependency measurement.

Session Manager: maintains sessions, problem state, attempts, and history.

Analyzer Agent: analyzes code, errors, attempts, and context.

Level Controller: chooses the appropriate amount of assistance.

Hint Agent: generates guidance consistent with the selected level.

LLM Router: selects the model and handles fallback.

Telemetry Recorder: stores interaction events.

Analytics Layer: aggregates session data.

Dashboard: displays progress, dependency, attempts, and AI usage.

6. Adaptive Assistance

The main learning mechanism is graduated assistance. The system should avoid revealing a complete solution when a smaller intervention can help the learner continue independently.

L1 – Directional: “Check the condition controlling when the loop terminates.”

L2 – Conceptual: “Think about what happens at the final valid index.”

L3 – Structured: “Compare the loop boundary with the valid index range.”

L4 – Detailed: Explain the boundary error and its effect.

L5 – Solution-oriented: Used only when detailed guidance is justified.

The Level Controller can consider failed attempts, previous hint level, error type, previous performance, and evidence of understanding.

7. AI Dependency Score

AACM uses a proposed AI Dependency Score (ADS) to summarize observable reliance on AI during programming sessions. It is a project-specific behavioral metric and is not intended to diagnose psychological dependency.

$$\text{ADS} = 0.30H + 0.20L + 0.20R + 0.15C + 0.15S$$

7.1 Interpretation

H represents normalized hint-request frequency, L represents normalized assistance level, R represents normalized assistance-request frequency, C represents normalized AI-dependent code-correction activity, and S represents the solution-disclosure indicator. Components are normalized before weighting and the resulting value is mapped to a 0–100 scale. Higher values indicate greater observed reliance on AI assistance in the measured interaction. The weights are initial design parameters and should be validated and refined using collected data.

8. Functional Modules

Authentication and User Management maintains learner accounts and sessions. The Problem Workspace contains the problem, editor, submission controls, attempt history, AI mentor, and model selector. The AI Mentor generates guidance appropriate to the selected assistance level. The Model Router abstracts Qwen, Gemini, and Grok and provides fallback. Telemetry and Analytics record submissions, results, hints, assistance levels, model use, response time, and completion data.

9. User Interface

Dashboard: overall learner status, skill trend, ADS, recent problems, attempts, and hints.

Workspace: problem, editor, AI mentor, model selector, hint level, and attempts.

Progress: detailed skill and dependency analytics.

Research/Evaluation: session metrics and comparison results.

Documentation: architecture, agents, scoring, routing, and methodology.

10. Typical Workflow

1. Learner selects a programming problem and starts a session.
2. Session Manager initializes the session and telemetry.
3. Learner submits code.
4. Analyzer Agent examines code, result, and previous attempts.
5. Level Controller selects an assistance level.
6. Hint Agent generates guidance.
7. LLM Router sends the request and activates fallback if needed.
8. Learner receives guidance and retries.
9. Telemetry is updated and analytics are recalculated.
10. Final session data is stored for progress and research analysis.

11. Technology Stack

Frontend: React.js, Vite, Tailwind CSS.

Routing: React Router.

Backend: Node.js, Express.js.

Database: MongoDB / Mongoose.

Authentication: JWT.

LLM Layer: Qwen, Gemini API, Grok API.

Deployment: Vercel frontend and Render/cloud backend.

Version Control: Git and GitHub.

12. Data Model

User: userId, name, email, skillLevel, createdAt.

Problem: problemId, title, difficulty, topic, description, testCases.

Session: sessionId, userId, problemId, startTime, endTime, status.

Attempt: attemptId, sessionId, code, result, runtime, timestamp.

AI Interaction: interactionId, sessionId, model, hintLevel, requestType, responseTime.

Telemetry: eventType, sessionId, timestamp, metadata.

Analytics: skill indicators, ADS components, aggregated metrics.

13. Security and Privacy

External LLM API keys must remain on the backend and must not be exposed in frontend code. User-submitted code should be treated as untrusted input. Research exports should use anonymized identifiers where possible. Only data required for the application's educational and research objectives should be retained.

14. Evaluation Methodology

Evaluation should consider both technical performance and educational behavior. Technical measures include response latency, model availability, fallback success, and response quality. Educational measures include problem completion, completion time, independent attempts, hints requested, direct-solution requests, and post-test performance.

A practical experiment can compare an unrestricted AI condition with the AACM condition. Both groups can receive comparable programming problems, while AACM uses graduated hints and records dependency-related telemetry. ADS can be compared across sessions, alongside completion and post-test performance.

15. Expected Outcomes

The expected outcome is a functional AI coding mentor that provides useful assistance without making complete solution generation the default. The system should adapt guidance, maintain interaction history, switch between configured models, and provide measurable learner analytics. Experimental evaluation will determine whether these mechanisms are associated with improved independent problem solving and changes in AI-assisted behavior.

16. Limitations

ADS is a proposed metric and requires empirical validation. LLM hints can be incorrect or pedagogically unsuitable. Telemetry cannot capture every aspect of learning or cognitive dependency. Results may vary with programming experience and problem difficulty. API availability and model changes can affect behavior. Small experimental samples may limit generalizability.

17. Future Enhancements

Learn ADS weights from validated experimental data; add learner-state prediction for assistance selection; automatically evaluate hint correctness and pedagogical quality; support more programming languages and online judges; add course-specific retrieval-augmented generation; conduct longitudinal studies; and personalize problem difficulty using demonstrated skill.

18. Conclusion

AACM is designed as an educational programming environment rather than a conventional code-generation chatbot. Its central idea is to regulate AI assistance according to learner behavior while collecting interaction data for learning and AI-reliance analysis. The combination of graduated hints, agent-based analysis, multi-model routing, telemetry, and the proposed AI Dependency Score provides a practical foundation for a final-year project and an empirical research study. The effectiveness of the framework should ultimately be established through controlled evaluation

rather than assumed from the architecture alone.

19. References

- [1] M. Kazemitabaar et al., "CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs," Proceedings of CHI, 2024.
- [2] "Enhancing Python learning with PyTutor: Efficacy of a ChatGPT-Based Intelligent Tutoring System in Programming Education," Computers and Education: Artificial Intelligence, 2024.
- [3] H. Zhang et al., "Generative AI dependency on programming among university students: a scale development and validation study," BMC Psychology, 2026.